SmokeStudio R2 — Reporte de QA & Seguridad

> ***Fecha:*** *2026-04-23 00:44*
> ***Analista:*** *Claude Code (Sonnet 4.6) + Skills: supabase, supabase-postgres-best-practices*
> ***Rama:*** *SmokeStudio.Rev01*
> ***Archivos analizados:*** *smokescan.js (1599 líneas), main.js (196), smokescan.html (1458), nosotros.html, servicios.html*

Resumen Ejecutivo

Dimensión	Estado	Incidencias
Seguridad	❌ Crítica	2 críticas, 3 altas, 2 medias
Backend / Supabase	⚠️ Parcial	3 problemas funcionales
QA / Bugs funcionales	❌ Deficiente	1 crash garantizado, 2 datos perdidos
UX / UI	⚠️ Parcial	6 observaciones
Accesibilidad	⚠️ Parcial	1 bloqueador de teclado

Vulnerabilidades de Seguridad

[CRÍTICA] XSS — Input del usuario inyectado en innerHTML

Archivo: `smokescan.js:1486`

```
// VULNERABLE
chatWindow.innerHTML += `<div ...>${txt}</div>`;
```

El texto del chatbot que escribe el usuario (`txt`) se inserta directamente en `innerHTML` sin ninguna sanitización. Un mensaje como `<img src=x onerror=alert(document.cookie)>` ejecuta JavaScript arbitrario en el contexto del sitio.

Impacto: Robo de sesión, ejecución de código malicioso, phishing dentro del sitio.

Fix requerido:

```
// CORRECTO
const div = document.createElement('div');
div.textContent = txt;
// aplicar estilos como propiedades de style...
chatWindow.appendChild(div);
```

[ALTA] XSS — Contenido de Gemini renderizado sin sanitizar

Archivos: `smokescan.js:829-874`

Los campos `cat.name`, `cat.desc`, `step.title`, `step.description` de la respuesta del proxy Gemini se insertan via template literals en `innerHTML`. Si el Edge Function es comprometido o la API devuelve contenido malicioso, se ejecuta JS en el cliente.

Fix requerido: Usar `DOMPurify.sanitize()` o asignar con `element.textContent` para campos de texto plano.

[ALTA] Bug Crítico — `getScoreStatus()` no definida

Archivo: `smokescan.js:1183`

```
const newResult = getScoreStatus(); // ReferenceError: getScoreStatus is not defined
```

Se llama en `applyChangesAndReanalyze()`, que se activa cuando el usuario edita respuestas desde el panel de resultados. Cualquier edición post-análisis produce un crash (`ReferenceError`) y congela la UI.

Fix requerido: Reemplazar con `generateSimulatedResult()`.

[ALTA] `updateBreakdown` definida dos veces

Archivo: `smokescan.js:132` y `smokescan.js:773`

La segunda definición (línea 773) sobrescribe silenciosamente la primera (línea 132) en runtime. La primera nunca se ejecuta. Aunque ambas son funcionalmente equivalentes, la duplicación es un indicador de deuda técnica y puede causar regresiones al modificar una sin la otra.

Fix requerido: Eliminar la definición de la línea 132 y dejar únicamente la de línea 773 (que tiene mejor lógica de prefix).

[MEDIA] `saveAnalysis()` nunca se llama

Archivo: `smokescan.js:1295-1304`

La función que persiste análisis en Supabase existe pero ningún flujo la invoca. Los análisis completados **nunca se guardan en la base de datos**.

Fix requerido: Llamar `saveAnalysis()` al final de `showResults()`.

[MEDIA] `SUPABASE_ANON_KEY` hardcodeada en el repositorio

Archivo: `smokescan.js:1213`

```
const SUPABASE_ANON_KEY = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...';
```

El anon key es público por diseño de Supabase y seguro siempre que RLS esté correctamente configurado. Sin embargo, tenerlo versionado en git dificulta la rotación de claves y expone el Project Ref públicamente.

Recomendación: Inyectar la clave desde un meta tag generado por el build/deploy pipeline, o mantenerla en un archivo `.env` excluido del repo.

[MEDIA] `raw_user_meta_data` usado para nombre de usuario

Archivo: `smokescan.js:1282`

```
name: profile?.name || session.user.user_metadata.full_name,
```

`user_metadata` es editable por el usuario (campo inseguro para autorización según el skill de Supabase). Aunque aquí solo se usa para mostrar el nombre, si ese nombre se renderiza vía `innerHTML` en algún punto, se convierte en vector de stored XSS.

Backend / Supabase

Problema 1: Parsing frágil de respuesta Gemini

Archivo: `smokescan.js:577-579`

El código asume que el proxy retorna el shape exacto de la API de Gemini (`candidates[0].content.parts[0].text`). Si el Edge Function wrappea la respuesta, falla silenciosamente y cae al resultado simulado sin registrar el error real.

Recomendación: Agregar logging del error real en el catch, y documentar el contrato del Edge Function (qué shape devuelve el proxy).

Problema 2: Sin validación de tamaño de prompt en el proxy

El Edge Function `gemini-proxy` recibe `{ prompt: string }` sin límite de tamaño. Un actor malicioso puede enviar prompts masivos y generar costos elevados en Gemini API.

Recomendación: Agregar en el Edge Function:

```
if (prompt.length > 4000) {
  return new Response('Prompt too long', { status: 400 });
}
```

Problema 3: RLS — verificar políticas en las 3 tablas

Según el skill `supabase-postgres-best-practices` , las tablas expuestas deben tener RLS activado. Para este proyecto:

Tabla	SELECT	INSERT	UPDATE	DELETE
`users`	`auth.uid() = id`	Solo sistema	Solo sistema	No
`analyses`	`auth.uid() = user_id`	`auth.uid() = user_id`	No	No
`leads`	Solo admin	Autenticado	No	No

QA — Bugs Funcionales

#	Severidad	Descripción	Archivo	Línea
1	**CRÍTICO**	`getScoreStatus()` undefined — crash al editar después de ver resultados	smokescan.js	1183

2	**ALTA**	`saveAnalysis()` nunca llamada — análisis no se persisten	smokescan.js	1295
3	**ALTA**	`initReset()` no resetea `STATE.answers.step0` — datos de rol/recursos quedan sucios al reiniciar	smokescan.js	1338
4	**MEDIA**	Datos del Step 0 (recursos: capital, inversores, equipo) jamás incluidos en el prompt de Gemini — el usuario los llena sin efecto	smokescan.js	540
5	**MEDIA**	HTML inválido: `</section>` huérfano en línea 146 — sección hero se cierra dos veces	smokescan.html	146
6	**BAJA**	Typo en copy: "*contrar tu estragia*" → "*construir tu estrategia*"	smokescan.html	190
7	**BAJA**	Comentario incorrecto: `// New Firebase Auth initialization` en código que usa Supabase	smokescan.js	1586
8	**BAJA**	`navWrapper.classList` accedido sin null-check en `main.js` — crash si el elemento no existe en la página	main.js	7

UX / UI

#	Área	Observación	Impacto
1	Auth / Loading	No hay skeleton ni spinner visible durante `onAuthStateChange` — hay flash de UI no autenticada	Medio
2	Step 0	Usuario llena datos de recursos que no impactan score ni análisis — genera expectativa falsa	Alto
3	Chatbot	Input sin límite de longitud — texto muy largo rompe el layout del bubble	Bajo
4	Accesibilidad	`.ss-option` son `<div>` sin `role="button"` ni `tabindex` — no navegables por teclado (WCAG 2.1 AA)	Alto
5	PDF Export	Contenedor temporal usa `left:-9999px` — puede causar scroll horizontal en algunos navegadores	Bajo
6	Responsive	Card se comprime con `style.maxWidth` via JS inline — puede sobreescribir media queries en breakpoints intermedios	Medio

---

Priorización de Correcciones

Semana 1 — Bloqueadores críticos

1. Fix XSS en chatbot (`innerHTML` + input de usuario) → `textContent`
2. Fix `getScoreStatus()` undefined → reemplazar con `generateSimulatedResult()`
3. Conectar `saveAnalysis()` en `showResults()`

Semana 2 — Seguridad y datos

4. Sanitizar resultados de Gemini con `DOMPurify` o `textContent`
5. Verificar políticas RLS en Supabase (usuarios, análisis, leads)

6. Agregar validación de tamaño de prompt en el Edge Function

Semana 3 — Calidad y UX

7. Fix reset bug — incluir `step0` en `initReset()`
8. Incluir datos de Step 0 en prompt de Gemini o eliminarlos del formulario
9. Agregar `role="button"` y `tabindex="0"` a `.ss-option`
10. Null-check en `main.js:7` para `navWrapper`
11. Fix HTML inválido (`</section>` huérfano)
12. Fix typo en copy
13. Eliminar comentario Firebase incorrecto
14. Eliminar definición duplicada de `updateBreakdown`

Generado por Claude Code — SmokeStudio.Rev01 — 2026-04-23 00:44